

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Asignatura: Sistemas Operativos – CAO503
Profesora: Mirella Herrera

PROYECTO: Sincronización entre Procesos

Objetivo

Poner en práctica los conocimientos adquiridos sobre el manejo de la concurrencia entre procesos, utilizando herramientas para la sincronización y comunicación, aplicadas en la resolución de problemas.

Condiciones

1. El proyecto se desarrollará en grupos de **mínimo tres (03) y máximo tres (03)** participantes.
2. **Cada grupo deberá desarrollar un problema a su elección.**
3. El código deberá ser desarrollado en **Lenguaje C**, intradocumentado y la herramienta de sincronización entre procesos es semáforos.
4. Copia de soluciones no serán revisadas y el grupo perderá la calificación del proyecto.
5. El proyecto deberá ser subido al drive con envío del mensaje al correo en fecha tope: **domingo 04 de octubre de 2026 a las 11: 59 pm**
6. La defensa del proyecto será el **miércoles 07 de octubre de 2026 a partir de las 8am.**

Pautas para la Entrega

1. Crear y compartir con sistemasoperativosfacytuc@gmail.com una carpeta en **Google drive** denominada **Proyecto_SO_Sem1_2026**, con los siguientes archivos:
 - a) Código en lenguaje C, makefile y README.md.
 - b) Video de la ejecución con los distintos casos de prueba y sus resultados, contemplando datos de prueba para ejecuciones correctas y casos de borde. Debe colocar banderas en la ejecución para reconocer los caminos de las distintas pruebas.
 - c) Archivo denominado Documentación.PDF (letra tipo Calibri tamaño 11 e interlineado sencillo), explicando su análisis, procesos, secciones críticas, recursos críticos, escenarios de concurrencia, así como la utilidad y aprendizaje con el proyecto. Es importante señalar, que todas las asunciones que realicen en la interpretación de las diferentes problemáticas, deben ser expresadas en el documento y recuerden que el paradigma de programación es concurrente, por lo tanto, no se puede restringir el número de procesos ni el orden en que éstos se entrelazan o ejecutan.
2. Enviar un mensaje al correo electrónico sistemasoperativosfacytuc@gmail.com, indicando:
 - a) Enlace a la carpeta
 - b) Nombres, apellidos y número de cédula de cada integrante del grupo

Problema 1: Motor de Emparejamiento Financiero de Alta Frecuencia (High-Frequency Trading)

En una bolsa de valores de activos digitales, se está diseñando el núcleo de su nuevo motor de emparejamiento de órdenes (Matching Engine). Este sistema debe procesar miles de transacciones de compra y venta de forma concurrente, garantizando la integridad absoluta de los fondos y evitando cualquier condición de carrera que pueda resultar en desbalances financieros.

Este sistema central dispone de:

- (01) Libro de órdenes compartido (Order Book),
- (04) Hilos receptores de órdenes (Brokers),
- (02) Motores de emparejamiento (Matchers) que ejecutan las transacciones,
- (01) Hilo auditor de balances, y
- (01) Hilo publicador (Ticker) que transmite los precios actualizados.

La dinámica del sistema se define de la siguiente forma:

- Los **Brokers** reciben órdenes (compra/venta) y las insertan en el Libro de Órdenes. Múltiples Brokers intentarán escribir simultáneamente, lo que requiere un control de acceso estricto.
- Los **Matchers** leen el Libro de Órdenes continuamente. Si encuentran una coincidencia (match) entre una orden de compra y una de venta, bloquean temporalmente los fondos de ambas partes, ejecutan la transacción, y eliminan las órdenes del libro.
- **Auditor (Lector)**: Se activa periódicamente para pausar temporalmente las modificaciones de los Matchers mediante un semáforo de control, sumar los balances y verificar la integridad de los fondos. No requiere buffer secundario.
- **Ticker (Lector Pasivo)**: Lee de forma concurrente el último precio ejecutado y lo imprime por pantalla sin bloquear la operación general.

Se requiere contabilizar, auditar y documentar:

- a) Volumen total de órdenes procesadas exitosamente por cada Matcher.
- b) Tiempos máximos de latencia de los Brokers al intentar escribir en el Libro de Órdenes (medición de contención).
- c) Número de veces que el buffer secundario de los Brokers superó su capacidad durante una auditoría.

- d) Demostración de al menos 3 casos de prueba con inyección de retardos (sleep()) para comprobar que los semáforos evitan condiciones de carrera.
- e) Log de simulación con 500 transacciones exitosas sin discrepancias financieras.
- f) Asistencia opcional de herramientas de IA para guiar la depuración y estructuración de las pruebas.
- g) La simulación debe evitar el problema clásico de los "Lectores y Escritores" con inanición (starvation) de los escritores, asegurando que los Auditores no impidan el trabajo de los Matchers indefinidamente.

Problema 2: Sistema de Enrutamiento y Prioridad de Emergencias en una Ciudad Inteligente (Smart City)

El departamento de tráfico de una metrópolis está simulando un nuevo algoritmo para el control automatizado de intersecciones críticas y la gestión de flotas de emergencia (ambulancias y bomberos). El objetivo es reemplazar los semáforos de luz tradicionales por un sistema de coordinación de red vehicular donde los vehículos "reservan" su paso por la intersección de forma concurrente.

La cuadrícula de simulación dispone de:

- (05) Intersecciones de alta densidad (representadas como recursos compartidos),
- (20) Vehículos autónomos regulares generados dinámicamente,
- (02) Vehículos de emergencia con prioridad absoluta, y
- (01) Orquestador Central de Tráfico.

La dinámica del sistema se explica de la siguiente forma:

- **Cada vehículo es un hilo independiente.** Para cruzar una intersección, el vehículo debe solicitar acceso al semáforo de dicha intersección.
- **Las intersecciones tienen una capacidad máxima** (ej. solo pueden procesar 2 vehículos no colisionantes a la vez).
- **Vehículos de emergencia:** Cuando un hilo de emergencia se acerca a una intersección, requiere que la intersección se "vacíe" y no se admitan más vehículos regulares hasta que este pase.
- **Problema de Inversión de Prioridad:** El diseño debe evitar explícitamente que un vehículo de emergencia quede bloqueado esperando a un vehículo regular que, a su vez,

está bloqueado por otro recurso. Los estudiantes deben implementar herencia de prioridades o protocolos similares.

Se requiere contabilizar, auditar y documentar:

- a) Tiempo de viaje total promedio de los vehículos regulares vs. los vehículos de emergencia (demostrando la efectividad de la prioridad).
- b) Contador de "Deadlocks evitados" en las intersecciones (por ejemplo, el escenario de 4 vehículos llegando exactamente al mismo tiempo a un cruce de 4 vías).
- c) Número de vehículos regulares encolados debido al paso de vehículos de emergencia.
- d) Ejecución de un script de prueba que lance 100 vehículos concurrentes para demostrar la ausencia de interbloqueos (*deadlocks*) o fallos de segmentación (*Segmentation Fault*).
- e) Todos los procesos deberán emitir mensajes estructurados en consola para que un script externo pueda leerlos y validar que las leyes de tránsito del sistema nunca fueron violadas por la ejecución concurrente.